

How the Love Button works

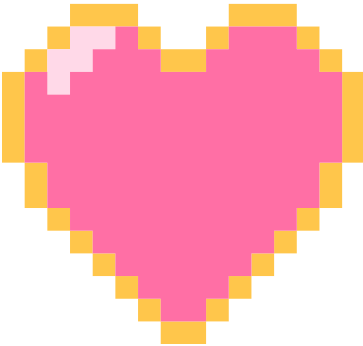
Tap a widget on your home screen and her phone buzzes with its own sound. This is how that works: first in plain words, then the technical detail.

4 messages

2 phones

1 server

£0 a month



Idle

click the button! (• ω •)



Sending

on its way to her 0o0



Sent

traveling in the interwebs (• ε •)



Delivered

it buzzed her phone :3



Seen

she looked at it (>^o^>)

Contents

1	The big picture	3
2	One tap, end to end	5
3	The server	7
4	Talking to Google	9
5	The app	11
6	Receiving a message	12
7	The ladder and the bubble	13
8	Widgets and pixel art	15
9	Security	16
10	Reliability and limits	17
G	Glossary	18

How to read it. Each chapter opens with a pink *In plain words* box; read only those and you'll still understand the app. Below it come *Under the hood* (how it really works) and *Why* (the reason for a choice).

Underlined words are explained in the glossary at the back; on the web, tap one to see its meaning in place. The sender's phone is **Phone A**, the recipient's is **Phone B**; both run the same app. Code excerpts are real, trimmed; notes after ← were added.

1

OVERVIEW

The big picture

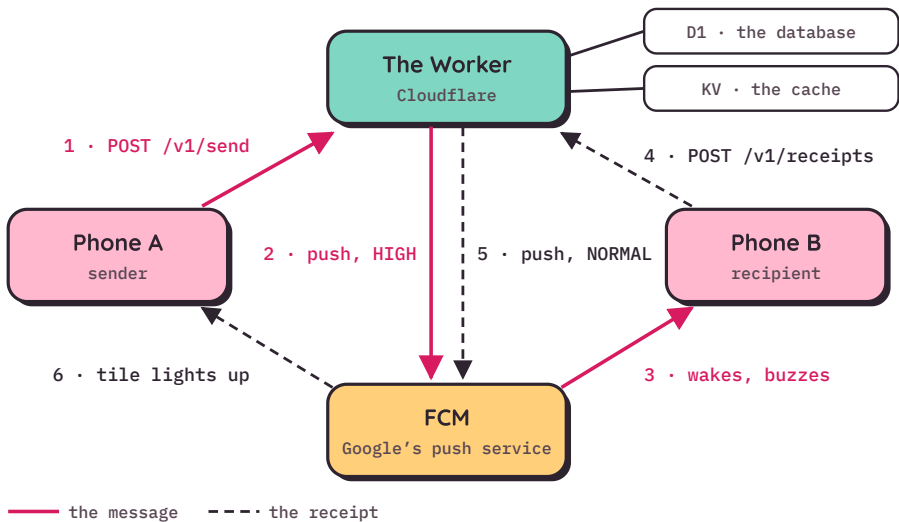
□ IN PLAIN WORDS

The Love Button sends one of four fixed messages to the other phone: “I love you”, “Thinking of you”, “Miss you” or “Call me when you can”. Each has its own home-screen widget, pixel picture and sound, so she knows which one it is without looking. The sender’s widget shows the message travelling and turns gold once she’s seen it.

Phones can’t reach each other directly: they have no fixed address and are usually asleep. So a small server that both can always reach carries the message, and it asks Google’s push service, the only thing that can wake a sleeping Android phone, to deliver it.

■ UNDER THE HOOD

- Phones sit behind NAT: they can connect out to the internet but nothing can connect in. Doze also cuts idle apps off the network. The one line that stays open into an Android phone is Google Play Services’ connection to Google, and FCM is how a server sends down it.
- Using FCM needs a Google service-account private key that can push to any phone in the Firebase project. Anyone can unpack an APK, so the key can’t go in the app; it lives only on the server.



Phones only ever talk *up* to the Worker, over HTTPS, and only ever hear back *down* through FCM.

PLAYER	RUNS ON	KNOWS
Phone A, Phone B	The two phones, same app	Its own bearer token, which person it is, the partner's name, the latest message
The Worker	Cloudflare	Every phone's push address, seven days of messages, the Google key
FCM	Google	Only the push itself: a message number, an id, a name, a time

WHY IT'S BUILT THIS WAY

- **The push carries a number, never words.** Her phone gets `msg_id: 3` and looks up the text, picture and sound itself. The words never pass through Google.
- **No request names the recipient.** The server works it out as `3 - sender's number` (people are 1 and 2), so there's nothing to fake: even a stolen token can only message its own partner.

PART BUILT WITH

The app Kotlin, Jetpack Compose (screens), Glance (widgets), WorkManager (background jobs), DataStore (storage), OkHttp (network), Firebase Messaging (receiving pushes)

The
server

TypeScript, Hono, Cloudflare Workers, D1 (database), KV (cache)

Cost

Free tiers only: £0 a month, no card on file

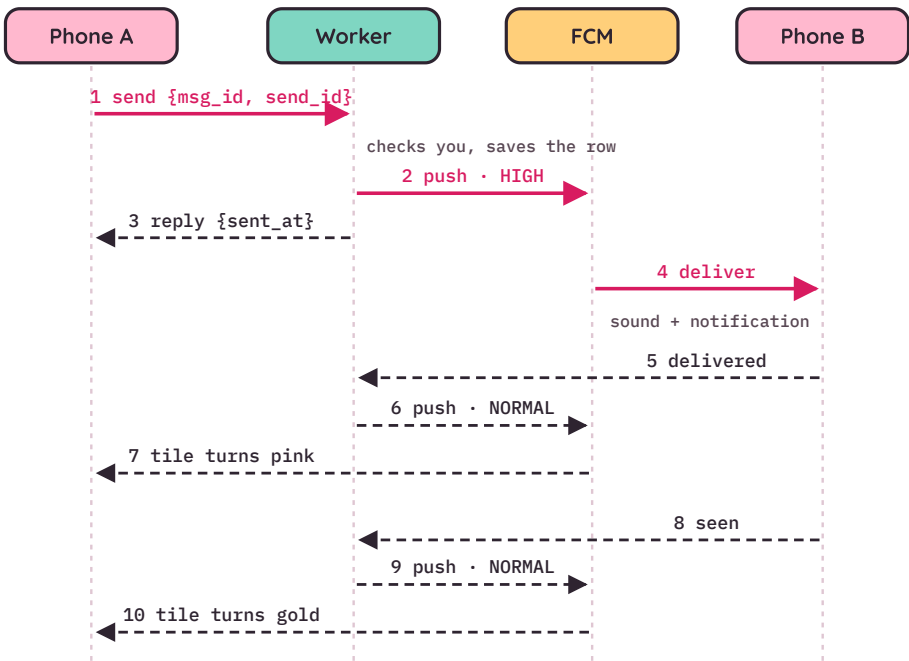


END TO END

One tap, end to end

□ IN PLAIN WORDS

You tap “Miss you”. Your phone gives the tap an id and tells the Worker “message 3”. The Worker checks it’s you, writes the message down and asks Google to wake her phone. Her phone plays the Miss-you sound and reports “delivered”, and your widget turns pink. When she looks at her phone it reports “seen”, and your widget turns gold. It normally takes a few seconds.



Time runs downwards. Step 7 travels a different route from step 3, so on a slow connection it can reach Phone A first.

- 1 **PHONE A** **Tap.** `SendAction` shows the half-filled heart, then `beginSend()` makes a `send_id` (a UUID), saves it with the tap time, and queues two jobs: `SendWorker` (waits for internet) and `TimeoutWorker` (waits 20 seconds).

- 2 **PHONE A** **Send.** `SendWorker` notes which widget the `send_id` belongs to, then sends `POST /v1/send` with `{"msg_id":3,"send_id":"..."}` and the phone's bearer token. No recipient.

- 3 **WORKER** **Who is it?** It hashes the token with `SHA-256` and finds the phone's row. That, and only that, decides the sender.

- 4 **WORKER** **Check and save.** Under 500 sends this hour, `msg_id` is 1-4, `send_id` is a UUID; otherwise an error status code (429 or 400). It saves the message with `to = 3 - from` and the server's clock, `sent_at`.

- 5 **WORKER** **Push.** It asks FCM for a data-only, HIGH-priority push to each of her phones, then replies to Phone A with `sent_at`. Phone A's tile turns crimson: *sent*.
-
- 6 **PHONE B** **Buzz.** Play Services wakes the app; `PushService` posts the notification on the "Miss you" channel, which plays its sound, then reports *delivered*. If she's looking at her phone it also reports *seen*; if not, it waits for her next unlock.
-
- 7 **WORKER** **Receipt.** `POST /v1/receipts`: only the recipient may acknowledge (403 otherwise). It records the time and pushes the receipt to the sender at NORMAL priority.
-
- 8 **PHONE A** **The tile climbs.** Pink for delivered, gold for seen, but only within 20 seconds of the tap. Gold stays for 4 seconds, then the tile rests.
-

WHY IT'S BUILT THIS WAY

- **The phone makes the id, not the server.** The receipt can overtake the server's reply. Because Phone A wrote down which tile the id belongs to before sending, it always knows which tile to light.

SERVER SIDE

3 The server

IN PLAIN WORDS

The Worker is about 960 lines of code that Cloudflare runs whenever a request arrives. It holds the Google key, decides who may send what to whom, and is the only place anything is stored outside the two phones.

UNDER THE HOOD

- **No machine to run.** Cloudflare runs the code on workerd, built on V8, in a lightweight isolate, in the data centre nearest the caller, only while a request

lasts. It gets web-standard tools (`fetch` , `WebCrypto`), not `Node.js`. The free plan allows 100,000 requests a day and 10 milliseconds of `CPU` per request.

- **Routing.** Hono sends each request to the function that handles it. Every reply is `JSON`, and every error has one shape: `{ "error": "rate_limited", "message": "..."} .`
- **No passwords.** `wrangler.toml` hands the code its database and cache as ready-made objects (bindings: `env.DB` , `env.TOKEN_CACHE`). Public settings (the Firebase project id, the two names) are plain vars. The three secrets (two enrolment codes and the service-account key) are uploaded with `wrangler`, stored encrypted, and can never be read back.

The database

D1 is `SQLite` run by Cloudflare. It has two tables:

TABLE	COLUMNS
devices	One row per phone: <code>id</code> , <code>person</code> (1 or 2, enforced by a <code>CHECK</code> constraint, so a third person is impossible), <code>auth_hash</code> (SHA-256 of its token), <code>fcm_token</code> (its push address; empty for a send-only device), <code>label</code> , <code>created_at</code> , <code>updated_at</code> .
sends	One row per message: <code>id</code> (the phone's <code>send_id</code>), <code>from_person</code> , <code>to_person</code> , <code>msg_id</code> , <code>sent_at</code> , <code>delivered_at</code> , <code>seen_at</code> .

Times are epoch seconds. A cron job deletes messages older than 7 days every night at 03:17 UTC: the rows are only needed for receipts and the hourly limit, not as a history. KV holds two short-lived things: the Google access token (55 minutes) and a count of enrolment attempts per `IP` address (one hour).

The six endpoints

ENDPOINT	NEEDS	DOES
GET /health	nothing	Answers <code>{ "ok": true } .</code>
POST /v1/enroll	a code	Trades an enrolment code for a 256-bit bearer token, sent once; only its hash is kept. 5 tries an hour per IP address.
POST /v1/devices	token	Saves the phone's new push address.
DELETE /v1/devices	token	Deletes the phone (the app has no button for this yet).
POST /v1/send	token	Sends a message (Chapter 2). Limit: 500 an hour per person.

ENDPOINT	NEEDS	DOES
POST /v1/receipts	token	Records delivered or seen. Recipient only.

WHY IT'S BUILT THIS WAY

- Identity only comes from the token. The `requireDevice` middleware hashes it and looks it up; nothing ever reads a sender from the request body.
- Nothing can happen twice. A message is saved with `INSERT OR IGNORE`, so a repeated id is refused (409) instead of overwritten. A receipt uses `COALESCE` (“keep the time already recorded”), so repeating it changes nothing and the state never goes backwards.
- No SQL injection. Values are passed separately from the SQL as placeholders (`?`), never pasted into it.

4 SERVER ↔ GOOGLE

Talking to Google

IN PLAIN WORDS

To wake her phone, the Worker asks FCM to deliver a small message. Google only accepts that from the project’s owner, so the Worker signs a note with its private key, swaps it for a pass that lasts an hour, and shows the pass with every push.

IDENTITY	WHAT IT IS	KEPT
google-services.json	The app’s identity: project id and an API key that only works for this app.	Inside the APK. Not secret.
Service-account key	The server’s identity: an RSA private key that can push to any phone in the project.	Only as a Worker secret.
Registration token	A phone’s push address, made by Play Services. It can change.	In <code>devices.fcm_token</code> .

UNDER THE HOOD

Getting the pass is OAuth 2.0’s signed-token flow:

1. Write a JWT saying who is asking (the key's email), for what (sending pushes) and until when (one hour).
2. Sign it with the private key using RS256 through WebCrypto.
3. Send it to `oauth2.googleapis.com/token`. Google checks the signature and returns an access token valid for one hour.
4. Keep it in KV for 55 minutes, and send it with each push:

```
POST https://fcm.googleapis.com/v1/projects/lovieapp-b4068/messages:send
Authorization: Bearer ya29... ← the access token
```

```
{ "message": {
  "token": "<her phone's registration token>",
  "data": { "type": "msg", "send_id": "8b1f5a4e-...", "msg_id": "3",
    "from_name": "Hubby", "sent_at": "1788274056" },
  "android": { "priority": "HIGH" } } }
```

■ WHY IT'S BUILT THIS WAY

- **The cache is required.** An RSA signature uses nearly the whole 10-millisecond CPU allowance, so signing on every send would break the free plan. With the cache, about one request an hour pays for it.
- **Data-only.** A notification message is drawn by Android itself and skips the app's channels, and with them the four sounds. A data message is handed to the app's code, which picks the channel.
- **HIGH for messages, NORMAL for receipts.** HIGH wakes a phone even in Doze. Receipts aren't urgent, and NORMAL saves battery.
- **Only UNREGISTERED deletes a phone.** Other errors can come from a bad request that fails for every phone at once. Deleting them all would lock her out until she enrolled again.

When Play Services gives a phone a new address, the app sends it to `/v1/devices` through a background job. FCM holds messages for a phone that's switched off (by default for up to four weeks), but a force-stopped app never receives them.

□ IN PLAIN WORDS

The app is the screen you open plus the four widgets. Behind them are background jobs that do the sending, and a few small files that remember what's going on.

■ UNDER THE HOOD

- **Start-up.** Android starts the app's process for anything: opening it, a push, a widget tap. Each time, `LoveButtonApp` creates the notification channels and starts listening for unlocks. The screen shows Enrol until the phone is enrolled, then Home.
- **Enrolling.** Once per phone: the app gets its push address from Play Services, sends it with the enrolment code to `/v1/enroll`, and saves the token, person number and partner's name.
- **Storage.** Five small DataStore files in app-private storage, kept out of cloud backups:

FILE	HOLDS
<code>love_button</code>	The token, which person this phone is, the partner's name.
<code>current_send</code>	The one latest message, shown on the app screen.
<code>pending_sends</code>	Which widget each recent message came from (about 20 seconds).
<code>unseen_sends</code>	Messages that arrived while nobody was looking.
<code>delivery_setup</code>	Whether you confirmed the Xiaomi settings.

BACKGROUND JOB	WAITS FOR	DOES
<code>SendWorker</code>	internet	Sends the message and updates the widget.
<code>TimeoutWorker</code>	20 seconds only	Turns a widget grey if it's still waiting.
<code>ReceiptWorker</code>	internet	Reports delivered or seen.
<code>RegisterTokenWorker</code>	internet	Sends a new push address.

WHY IT'S BUILT THIS WAY

- **Everything goes through WorkManager.** Xiaomi kills app processes routinely (process death). A request started straight from a tap would vanish; a WorkManager job is saved to disk and finishes later.
- **One attempt per message.** A failed send asks for a retry, but WorkManager's first retry comes after 30 seconds, when the 20-second window has closed, so it's dropped. A tap only counts while the sender is still waiting for it.

6 PHONE B Receiving a message

IN PLAIN WORDS

When the push reaches her phone, it plays that message's sound with a notification and tells the server "delivered". As soon as she's actually looking at her phone, it tells the server "seen".

ID	MESSAGE	CHANNEL	SOUND	WIDGET
1	I love you	msg_1	love.ogg	
2	Thinking of you	msg_2	thinking.ogg	
3	Miss you	msg_3	miss.ogg	
4	Call me when you can	msg_4	call.ogg	

UNDER THE HOOD

- **Channels.** Since Android 8, a notification's channel decides its sound. The app has four, created at every start-up (creating an existing one does nothing), each with high importance and its sound:
`android.resource://com.lovebutton.app/raw/miss`
- **The notification.** Title: the sender's name. Text: the message. A white heart icon, tinted crimson; a new id each time so several messages stack. On

Android 13 and newer it needs the notification permission.

- **Delivered** is reported only after the notification has been posted.
- **Seen** means she looked at her phone, not that she tapped the notification: the screen is on *and* unlocked (not on the lock screen). If so it's reported at once. If not, the message waits in `unseen_sends` and `UnlockReceiver` reports it the next time she unlocks.

THE ONE THING THAT CAN'T BE UNDONE

Android never lets an app change a channel's sound after the channel is created; the only way is to delete it, which resets her settings. So the four sounds were final before the channels first existed.

WHY IT'S BUILT THIS WAY

- The sound is addressed by its file name, because resource numbers change between builds while a channel keeps its sound address forever.
- The unlock listener is registered in code: since Android 8, the unlock broadcast isn't delivered to listeners declared in the manifest. If Android kills the app before she unlocks, no "seen" is sent: silence, never a false "seen".

PHONE A • BOTH PHONES

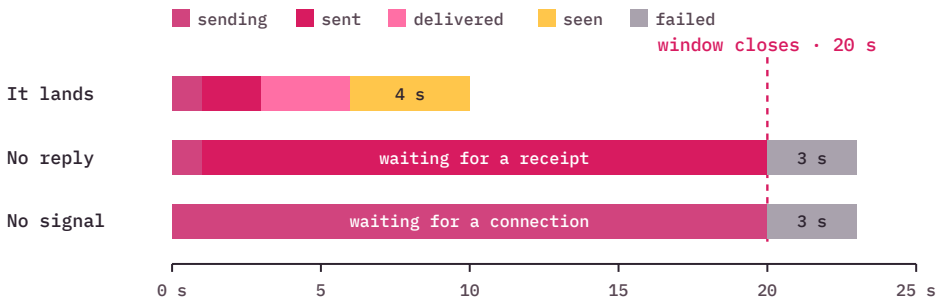
The ladder and the bubble

IN PLAIN WORDS

The sender's widget climbs a ladder: outline, half-filled, crimson, pink, gold. If nothing comes back within 20 seconds it turns grey instead. The app screen shows the same states, animated, and both phones agree on one "latest message".

	STATE	MEANS	SHOWN FOR
	IDLE	Resting.	—
	SENDING	Tapped; no reply yet.	Until the request ends

	STATE	MEANS	SHOWN FOR
	SENT	The server has it.	Up to 20 seconds
	FAILED	Failed, or no reply in 20 seconds.	3 seconds
	DELIVERED	Her phone showed it.	Up to 20 seconds
	SEEN	She looked at her phone.	4 seconds



Three ways a tap can go. The 20-second window and the 3- and 4-second holds are fixed; when receipts arrive varies.

■ UNDER THE HOOD

- **Only forward.** Each state has a rank (IDLE 0, SENDING 1, SENT 2, FAILED 3, DELIVERED 4, SEEN 5), and a receipt is written only if it moves the rank up. Her two receipts can arrive in either order, and a late “delivered” must not turn gold back to pink.
- **The 20-second window** (`PENDING_WINDOW_MS`). The app screen shows anything still waiting after 20 seconds as grey. `TimeoutWorker` greys the widget; it has no network condition, so it also fires with no signal. `SendWorker` drops a request that only found a connection after the window. Late receipts are still recorded, but never light the widget.
- **One writer at a time.** Every change goes through `WidgetStateWriter`, which checks and writes in one step, and a timer only resets the picture it put there. That prevents a race condition.
- **The app screen remembers.** Where the widget goes back to rest, the app keeps showing how the latest message ended. It draws the icon pixel by

pixel so it can animate it (rising fill, a ripple, gold running round the edge).

The shared bubble

The app screen's card shows the latest message between you, whoever sent it, and both phones must agree on which that is. An arriving message takes the card if its server time (`sent_at`) is later. In a tie within the same second, the higher `send_id` wins. Your own message still in flight has no server time yet, so it gives way, then takes the card back when its reply arrives if what replaced it is older.

■ WHY IT'S BUILT THIS WAY

- Phone clocks differ, so only the server's clock is compared. It counts whole seconds, so ties are common, and the id tie-break gives the same answer on both phones (a total order).



PHONE SIDE

Widgets and pixel art

□ IN PLAIN WORDS

A widget isn't running anything: it's a picture your home screen draws for the app. So each state is a finished picture, and every picture is drawn by a script from a grid of letters in the code.

■ UNDER THE HOOD

- **Widgets.** The home screen (the `launcher`) is a separate app. A widget reaches it as `RemoteViews` sent across `IPC`: a few standard building blocks, with no custom drawing or animation. That's why the ladder swaps whole pictures.
- **Four widgets, one class.** The widget list shows four entries (each 2×2 cells), all using `MessageWidget` with a different message id. A tap writes `SENDING` and calls `beginSend()`. After a reboot, `BootReceiver` redraws them.
- **Pixel art.** `scripts/pixel_icons.py` holds each icon as a grid of letters (`x` solid, `.` empty) and turns it into the 20 widget pictures (4 icons × 5 states).

□ IN PLAIN WORDS

The code is public, so safety can't depend on it being secret. It depends on three rules the server always checks, tokens that are useless if copied, and three secrets that live only on the server.

■ UNDER THE HOOD

The three invariants, each with a test:

1. **The sender is the phone that proved who it is.** Nothing reads a sender from the request.
 2. **The recipient is worked out on the server**, as `3 - from_person`. No request can name one.
 3. **Only the recipient may acknowledge a message.** Otherwise a sender, who knows every `send_id`, could fake "seen".
- **Tokens** are 32 random bytes (256 bits), sent once and stored only as a SHA-256 hash, so a stolen copy of the database holds no working keys.
 - **Enrolment codes** are 48 hex characters, one per person, limited to 5 tries an hour, and checked with a constant-time comparison, so timing reveals nothing (a timing attack). Changing a code stops new enrolments but doesn't cut off an enrolled phone; for that you delete its row.
 - **On the phone**, the token sits in app-private storage and is never logged; the release build can't be inspected over USB. A pre-commit hook refuses any commit containing a private key or a secrets file.

The three real secrets (the service-account key and the two enrolment codes) live only as Worker secrets and in a password manager; the release keystore that signs the app is kept outside the repo. Not covered: someone holding her unlocked phone, and end-to-end encryption. The pushes pass through Google, but they only ever say "message 3".

□ IN PLAIN WORDS

Xiaomi's phone software, MIUI, kills background apps far more than normal Android. The app can't change those settings itself, so its Delivery setup screen shows where each one is.

SETTING	DEFAULT	WHAT IT BREAKS
Battery optimisation (Android)	On	Background work is delayed. The app asks to be exempt.
Autostart (Xiaomi)	Off for <u>sideloaded</u> apps	A push can't start the app; widgets aren't redrawn after a reboot.
Battery saver (Xiaomi)	"Restricted"	Background work is suspended. Set "No restrictions".
Lock in recents (Xiaomi)	Unlocked	The app is cleared when memory runs low.

The Xiaomi settings can't be read by an app, so you confirm them by hand, and the Home screen shows "Finish delivery setup" until you do.

Limits

- **Force stop:** nothing arrives until the app is opened again, and missed messages aren't replayed.
- **China ROM:** no Play Services, so no FCM. Both phones must run Xiaomi's Global or European (EEA) software.
- **No signal:** the widget greys at 20 seconds and the message is dropped, not sent late.
- **Her phone off:** she gets it when it's back on; your widget will already be grey.
- **No sign-out button:** a retired phone is removed on the server.

Access token

A pass from Google that lasts one hour and lets the Worker send pushes.

API key Application Programming Interface key

A code that identifies an app to a Google service. This one only works for this app, so it isn't secret.

APK Android Package

The file an Android app is installed from.

App-private storage

Files only one app can read; other apps can't open them.

Bearer token

A long random key each phone sends with every request. Whoever holds it is trusted, like cash.

Binding

A database or store Cloudflare hands to the code as a ready-made object, so no password is needed.

Bit

A single 0 or 1. A 256-bit token has more possible values than could ever be guessed.

Broadcast

An announcement Android sends to apps that are listening, such as "the phone was unlocked".

CHECK constraint

A rule the database itself enforces. Here: `person` must be 1 or 2.

Compose Jetpack Compose

Android's modern way of building screens in Kotlin code.

Constant-time comparison

Comparing two secrets in a way that always takes the same time, so the time gives nothing away.

CPU Central Processing Unit

The chip that does the computing.

Cron

A schedule for running a job at set times, such as 03:17 every day.

D1

Cloudflare's database service, based on SQLite.

Data message

A push handed to the app's own code, which decides what to show.

DataStore

An Android library for saving small bits of data in private files.

Doze

Android's deep sleep for an idle phone: apps lose network access until a short wake-up window.

End-to-end encryption

Encryption only the sender and recipient can read. Not used here; the messages are just numbers.

EEA European Economic Area

The EU plus Iceland, Liechtenstein and Norway. Xiaomi's EEA software includes Google Play Services.

Enrolment code

A long secret code, one per person, typed once on each phone to get its token.

Epoch seconds

Time counted in seconds since 1 January 1970, UTC.

FCM Firebase Cloud Messaging

Google's service for sending pushes to phones; the only way to wake a sleeping Android phone.

Firebase

Google's set of services for apps. This app only uses Cloud Messaging.

Force stop

A button in Android's settings that kills an app and blocks it, including its pushes, until it's opened again.

Git hook

A script git runs automatically, such as before every commit.

Glance

An Android library for building widgets in Compose style.

Google Play Services

Google's system software on Android phones. It keeps the always-open connection pushes arrive through.

Hex hexadecimal

Numbers written with 0–9 and a–f. Each character is 4 bits.

Hono

A small framework that sends each request to the code that handles it.

HTTPS HyperText Transfer Protocol Secure

The encrypted way web requests travel, so nobody in between can read or change them.

Invariant

A rule that must hold every single time.

IP address Internet Protocol address

The number that identifies a device on the internet.

IPC Inter-Process Communication

How two separate apps on the same phone pass messages.

Isolate

A small sealed-off space where one piece of code runs, able to start in milliseconds.

JSON JavaScript Object Notation

A text format for data: `{"name": value}` .

JWT JSON Web Token

A small signed note that proves who you are.

Keyguard

Android's name for the lock screen.

Keystore

The password-protected file holding the key the app is signed with.

Kotlin

The programming language of Android apps.

KV key-value

Cloudflare's simple store: save a value under a name, optionally for a limited time.

Launcher

The app that shows your home screen and draws widgets.

Middleware

Code that runs before a request's handler, here to work out who is calling.

MIUI, HyperOS

Xiaomi's versions of Android; HyperOS is the newer one. Both save battery by killing background apps.

NAT Network Address Translation

How routers share one internet address between many devices. Devices behind it can connect out, but nothing can connect in.

Node.js

A common way to run JavaScript on servers. Workers aren't Node.

Notification channel

Since Android 8, a named group of notifications with its own sound. This app has four.

Notification message

A push Android shows by itself, without the app choosing how. Not used here.

OAuth 2.0 Open Authorization

The standard way one piece of software gets permission from another, using tokens instead of passwords.

OkHttp

An Android library for making web requests.

Private key

The secret half of a key pair: whatever it signs can be checked with the public half.

Process

A running app, with its own memory.

Process death

Android ending an app without warning to free memory or battery.

Race condition

A bug where the result depends on which of two things happens first.

Registration token

A phone's push address from Google. It can change.

RemoteViews

Android's way of describing a small screen area for another app to draw. Every widget is one.

ROM read-only memory

Here: the phone's installed system software. Xiaomi makes one per region.

RS256

Signing with RSA and SHA-256.

RSA Rivest-Shamir-Adleman

A public-key encryption system, named after its inventors. Signing with it takes a lot of computing.

Secret

A value Cloudflare gives the Worker, stored encrypted and never shown again.

Send-only device

An enrolled device with no push address: it can send as its person but never receives.

Server

A program that waits for requests and answers them.

Service account

A Google account for software rather than a person.

SHA-256 Secure Hash Algorithm, 256-bit

A one-way fingerprint: the same input always gives the same result, and it can't be reversed.

Sideload

Install an app from an APK file instead of an app store.

Signing

Stamping an APK with a key so Android can check who made it. Updates must use the same key.

SQL Structured Query Language

The language for reading and changing a database.

SQL injection

An attack where text sent to a server gets run as SQL.

SQLite

A small, widely used database.

Status code

The number that opens a server's reply: 200 means OK, 400 a bad request, 401 not signed in, 403 not allowed, 404 not found, 409 already used, 429 too many.

Timing attack

Guessing a secret by measuring how long a system takes to reject each guess.

Total order

An ordering where any two items can be compared, and everyone gets the same answer.

TypeScript

JavaScript with types, checked before it runs.

UNREGISTERED

Google's answer for a push address that will never work again.

UTC Coordinated Universal Time

The world's reference time, with no daylight saving.

UUID Universally Unique Identifier

A random 128-bit id, such as `8b1f5a4e-7c2d-...` . Two are never the same in practice.

V8

Google's JavaScript engine, used in Chrome and in Cloudflare Workers.

WebCrypto

The standard cryptography tools built into browsers and Workers.

Widget

A small live tile an app puts on the home screen.

Worker Cloudflare Worker

Code Cloudflare runs on demand in its data centres. Here, the Love Button's server.

workerd

The program that runs Cloudflare Workers, both live and in the tests.

WorkManager

Android's library for background jobs that must finish even if the app is killed.

wrangler

Cloudflare's command-line tool for deploying and managing Workers.
